

Teaching Notes: Understanding Low-Rank Adaptation (LoRA)

1 Core Idea of LoRA

Large Language Models (LLMs) such as GPT or BERT have millions or billions of parameters. Fine-tuning all parameters for a new task is expensive and memory-intensive.

Low-Rank Adaptation (LoRA) offers an efficient solution: instead of updating the full weight matrix W , we keep W frozen and learn two small matrices, A and B , such that:

$$\Delta W = AB \tag{1}$$

Then, during fine-tuning, the model uses:

$$W' = W + \Delta W = W + AB \tag{2}$$

Here, A and B are much smaller than W (typically rank $r \ll \min(d, k)$), meaning the number of trainable parameters is drastically reduced.

2 How LoRA Works

2.1 Freezing the Base Model

All pretrained weights W are frozen. They retain the general knowledge learned from massive datasets.

2.2 Injecting Trainable Low-Rank Matrices

We introduce two small matrices:

- $A \in \mathbb{R}^{d \times r}$
- $B \in \mathbb{R}^{r \times k}$

Only A and B are updated during fine-tuning.

2.3 Resulting Weight Update

The updated output becomes:

$$Y = (W + AB)X \tag{3}$$

The pretrained weight W produces the base output, while the low-rank term AB adds a small, task-specific adjustment.

3 Initialization of A and B

To ensure LoRA starts without altering the pretrained model, A and B are initialized so that $AB \approx 0$ initially.

- A is initialized to all zeros.
- B is initialized with small random Gaussian values.

Hence initially:

$$\Delta W = AB = 0, \quad W' = W \tag{4}$$

This guarantees that before training begins, LoRA behaves identically to the pretrained model.

3.1 Example

Let $W \in \mathbb{R}^{4096 \times 4096}$ and rank $r = 8$. Then:

$$\begin{aligned} A &\in \mathbb{R}^{4096 \times 8} \\ B &\in \mathbb{R}^{8 \times 4096} \end{aligned}$$

with initialization:

$$\begin{aligned} A &= 0, \\ B &\sim \mathcal{N}(0, 0.01) \end{aligned}$$

4 Training Process

During fine-tuning:

1. **Forward pass:** $Y = (W + AB)X$.
2. **Backward pass:** gradients flow only through A and B .
3. **Frozen base:** W remains unchanged.

After training, A and B encode the learned task-specific update.

5 Inference and Merging

At inference, the LoRA parameters can be merged back into W :

$$W' = W + AB \tag{5}$$

or kept separate for modular task switching.

6 Gradient Flow in A and B

Gradients are distributed as:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \Delta W} B^T, \tag{6}$$

$$\frac{\partial L}{\partial B} = A^T \frac{\partial L}{\partial \Delta W} \tag{7}$$

This shows that:

- A learns output adjustment directions.
- B learns input combination patterns.

7 How A and B Are Updated

During fine-tuning, gradients computed from the loss L are used to update A and B . The frozen weight W remains unchanged.

7.1 Gradient Computation

For each step of backpropagation, the gradients are computed as:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \Delta W} B^T, \tag{8}$$

$$\frac{\partial L}{\partial B} = A^T \frac{\partial L}{\partial \Delta W} \tag{9}$$

7.2 Parameter Update Rule

Using a standard optimizer such as Adam or SGD, the updates are applied as:

$$A \leftarrow A - \eta \frac{\partial L}{\partial A}, \quad (10)$$

$$B \leftarrow B - \eta \frac{\partial L}{\partial B} \quad (11)$$

where η is the learning rate.

7.3 Intuition

- A learns how to adjust the output directions for the task.
- B learns how inputs combine to produce those directions.

Initially, $A = 0$ so that $\Delta W = 0$. As training progresses, both A and B gradually learn the optimal low-rank correction AB that modifies W for the new task.

7.4 PyTorch Example

```
loss.backward()      # Compute gradients automatically
optimizer.step()     # Updates A and B only (W is frozen)
```

This process allows LoRA to fine-tune efficiently, updating only a tiny fraction of total parameters.

8 PyTorch Pseudo-Code

```
class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=8, alpha=16):
        super().__init__()
        self.r = r
        self.alpha = alpha
        self.scaling = alpha / r

        # Base weight (frozen)
        self.weight = nn.Parameter(torch.randn(out_features, in_features))
        self.weight.requires_grad = False

        # LoRA weights
        self.A = nn.Parameter(torch.zeros(out_features, r))          # zero init
        self.B = nn.Parameter(torch.randn(r, in_features) * 0.01)  # small random init

    def forward(self, x):
        delta = (self.A @ self.B) * self.scaling
        return x @ (self.weight.T + delta.T)
```

9 Summary of A and B

Component	Size	Initialization	Learns	Purpose
A	$d \times r$	zeros	Output adjustment	Controls output direction
B	$r \times k$	small random	Input combination	Determines input influence

10 Teaching Summary

LoRA creates two small matrices A and B that start with zero effect. During fine-tuning:

- A learns what kind of correction the model should make.

- B learns how inputs trigger those corrections.

Together, they form a small but powerful “patch” on top of the pretrained model.